

Week 2, Lecture 2 Handout: Typography, Colors, and Display Modes

🧠 Web Typography & Text Control

Typography dictates how readable and professional your website looks. In CSS, we control text using properties that handle size, thickness, and spacing:

- **font-family**: Sets the typeface of the text. Standard practice is to provide a fallback list (e.g., `font-family: Arial, sans-serif;`). If the browser cannot load Arial, it defaults to the system's clean sans-serif font.
 - **font-size**: Adjusts the scale of your text. While `px` (pixels) is fine for beginners, modern development prefers `rem` units because they scale smoothly based on user browser settings, improving accessibility.
 - **font-weight**: Controls thickness (e.g., `normal`, `bold`, or numeric values like `400` for regular and `700` for bold).
 - **line-height**: Controls the vertical spacing between lines of text. Setting this to a value like `1.5` prevents text blocks from looking cramped.
-

🧠 The CSS Color System

Web browsers read colors in three primary formats. Professional teams favor systems that allow transparency control:

1. **Hexadecimal (HEX Codes)**: Six-digit codes representing Red, Green, and Blue values starting with a hashtag (e.g., `#ffffff` is white, `#000000` is black, `#3498db` is light blue).
 2. **RGB / RGBA**: Stands for Red, Green, Blue, and Alpha (transparency). `rgba(52, 152, 219, 0.5)` creates a blue color with 50% opacity.
 3. **Color Keywords**: Simple built-in names like `red`, `blue`, or `tomato`. (*Avoided in professional settings due to lack of precision*).
-

📦 Master Concept: The `display` Property

In Lecture 1, students learned that everything is a box. In this class, they learn that there are different **types** of boxes. The `display` property determines how an element behaves relative to neighboring tags:

1. Block-Level Elements (`display: block;`)

- **Behavior:** Automatically starts on a brand-new line and stretches to fill the full width of its parent container.
- **Examples:** `<h1>` through `<h6>`, `<p>`, `<div>`, `<form>`, `<section>`.
- **Box Rules:** Honors `width`, `height`, `padding`, and `margin` rules fully.

2. Inline Elements (`display: inline;`)

- **Behavior:** Sits tightly side-by-side with other inline elements on the same line. It only takes up as much width as its content requires.
- **Examples:** `<a>`, `<strong>`, `<em>`, `<span>`.
- **Box Rules:** Ignores `width` and `height` settings. Vertical margins and padding will not push away neighboring block elements.

3. Inline-Block Elements (`display: inline-block;`)

- **Behavior:** A hybrid mode. Elements sit side-by-side on the same line like an inline tag, but they fully honor `width`, `height`, margins, and padding like a block tag.
 - **Best Use Case:** Excellent for styling clickable links (`<a>`) into large, spacious call-to-action buttons.
-

Homework Assignment: Enhancing Layout Contrast & Typography

Objective: Apply professional typography, typographic line spacing, custom web color harmonies, and modified display properties to your profile project.

Task Instructions

1. Open your project folder inside **VS Code** and open `style.css`.
2. Target your global `body` selector and set a professional line height (`line-height: 1.6;`) and color typography theme using a clean font fallback stack:

```
body {  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  color: #2c3e50;  
}
```

3. Target all form `<label>` tags. Since they are inline by default, change their display mode to block so they snap cleanly onto their own line directly above your input fields:

```
label {  
  display: block;
```

```
margin-bottom: 8px;  
font-weight: bold;  
}
```

4. Style your submission `<button>`. Change its layout mode to `display: inline-block;` or `display: block;`, remove the default browser outline border, add ample inner space (`padding: 12px 24px;`), and apply a vibrant, high-contrast background color (e.g., `#2ecc71`).
5. Change the button cursor property to a pointer to give users an immediate interactive visual hint:

```
button {  
  cursor: pointer;  
}
```

6. Save your updates, run **Live Server**, and check that your text blocks read comfortably and your labels stack perfectly above your inputs.

